INTERVIEW PORTFOLIO · 面试作品集

VISUALENGINEER

受治理的 AI 设计工程与交付系统

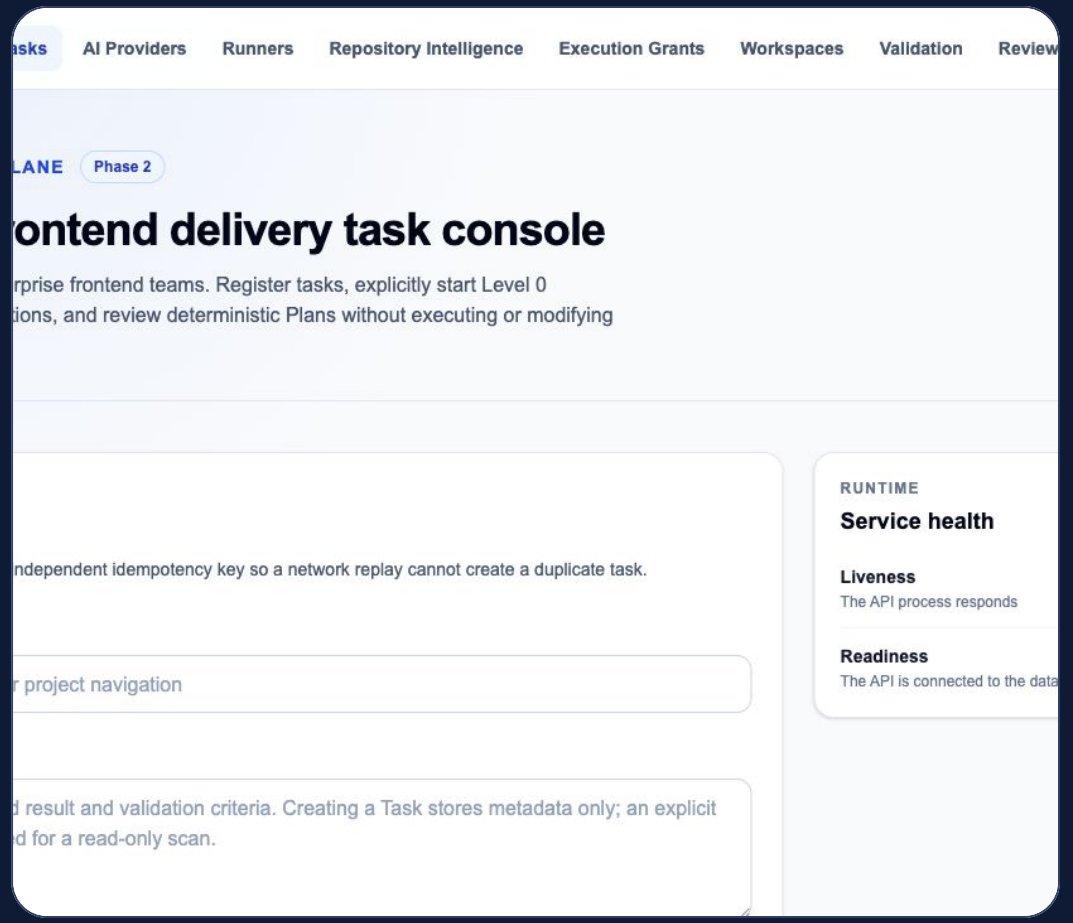
将 AI 规划、授权、执行、验证与证据审计拆成彼此独立、可审查的工程边界。

Plan ≠ Execution Grant

Runner Isolation

Evidence-first

github.com/dankk0220abc-prog/visual-engineer



AI 可以提出实施方案，但不应该自动获得修改代码仓库的权限。

一个“看起来正确”的 Plan，不等于 Workspace mutation、Runner authority、Review 或 Delivery 的授权。

01

提案 ≠ 权限

Provider 输出是类型化规划产物，进入受控生命周期，而不是直接变成工具调用。

02

Execution Grant 独立

只有获得独立范围授权的合格任务，Runner 才能进入受限工作区。

03

保留外部不确定性

派发后的模糊结果保持为 outcome_unknown，不靠自动重试伪造确定性。

04

Evidence 安全投影

公开事实不包含密钥、完整上游 ID、私有载荷或账单账户材料。

核心目标：把概率性的 AI Planning 与确定性的 Software Execution 之间的权限边界做成可验证的系统契约。

SYSTEM ARCHITECTURE • 系统架构

围绕受限 AI 提案构建的 Control Plane。



关键边界：Provider Layer 可以返回类型化 Plan，但不拥有任何直接 Runner 或 Workspace 权限。

W • WEB

Web Console

任务、Provider、Grant、Runner、Validation、Evidence 的浏览器入口。

A • API

Control Plane

状态机、策略、权限转换、Provider Invocation 与安全投影。

R • RUNNER

Independent Runner

消费封闭类型化任务和 Grant，不接受模型的任意 shell 权限。

E • EVIDENCE

GOVERNED WORKFLOW · 受治理工作流

Plan Approval 不等于 Execution Grant。

Provider 成功生成 Plan，只证明“方案已生成”；它不会自动产生执行、评审、交付或部署权限。

STEP 1

Provider Proposal

生成受约束的 Typed Plan，并通过系统策略评估。

AI Plan · Generated

→

AUTHORITY BOUNDARY

Human Approval

Plan 是否被接受，是独立决策；成功生成不等于批准。

Approval · Not decided

→

STEP 3

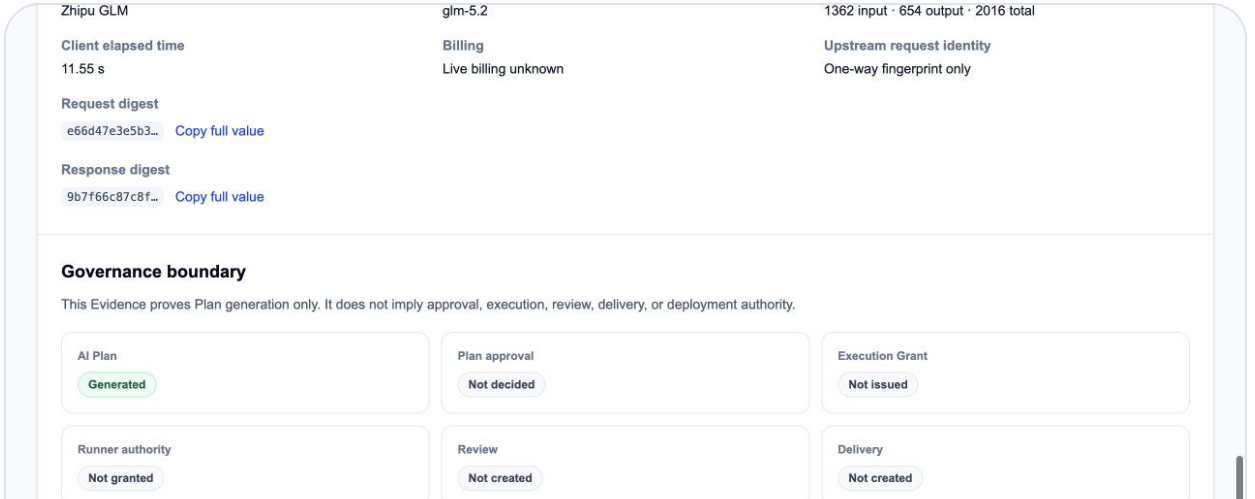
Execution Grant

只有独立、范围明确且合格的 Grant 才能让 Runner 进入变更路径。

Grant · Not issued

Provider 无法直接执行

- × 签发 Grant
- × 任意 shell
- × 修改 Workspace
- × 批准 Review
- × Authorize Delivery
- × Merge / Deploy



PROVIDER LAYER + BYOK

支持、连接和 Live Validation 是三件不同的事。

SAFE DEFAULT

Deterministic Fixture

离线、无网络、不可计费；默认 Demo 不需要真实 Provider Key。

无需凭据　Fixture 已验证

LIVE VERIFIED

Zhipu / GLM-5.2

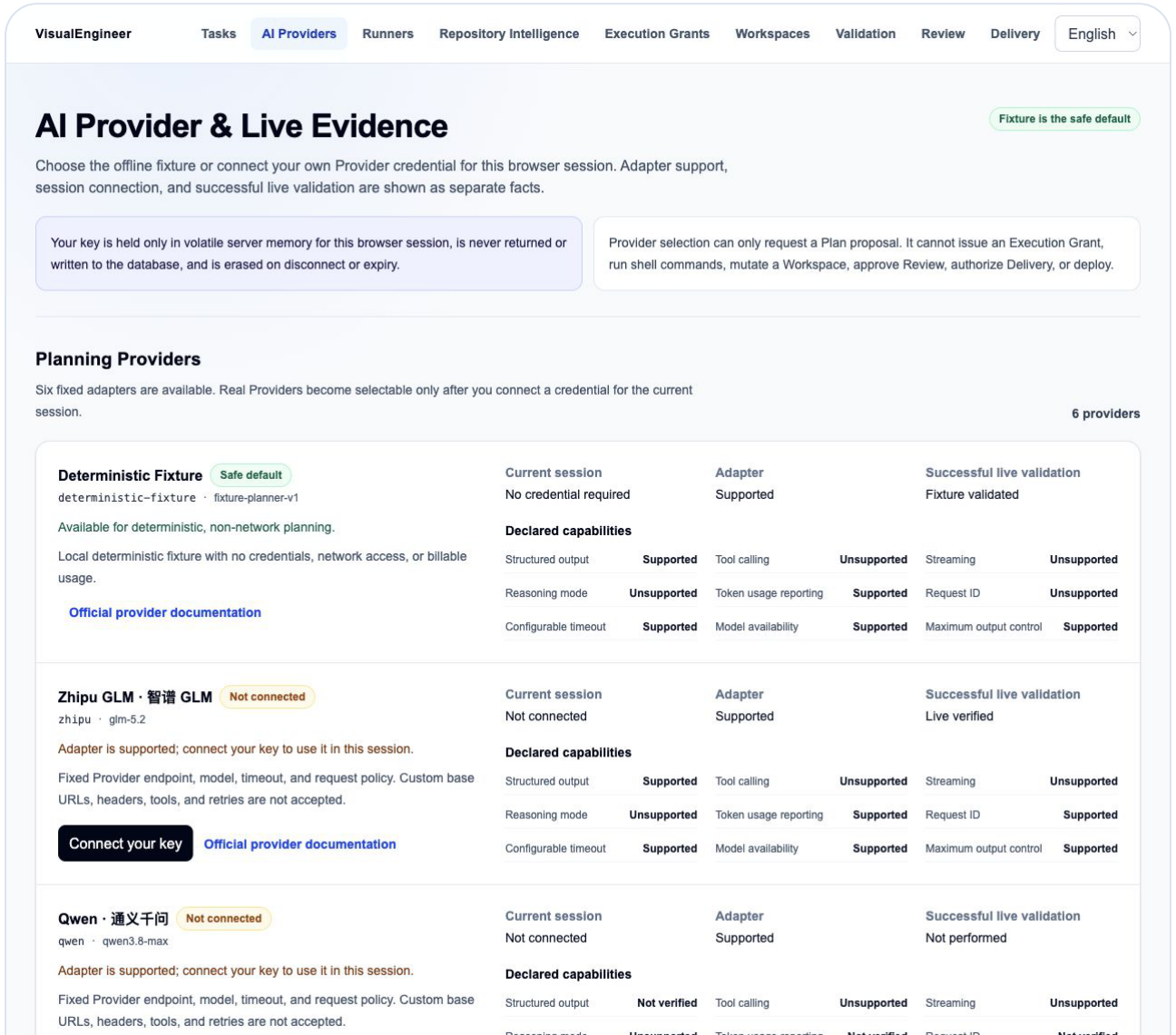
OTHER ADAPTERS

Supported ≠ Live validated

BYOK

会话范围的易失性凭据

普通用户不会 fallback 到维护者凭据。当前实现不被夸大为商业级多租户 Secret Vault。



SEALED LIVE VALIDATION • 已封存真实调用

一次受治理的 Zhipu / GLM-5.2 调用成功完成。

CORRELATION

3a9c354f7a58

安全 Evidence 关联标识

REPORTED USAGE

2,016 tokens

1,362 input + 654 output

MEASURED LATENCY

11.55 s

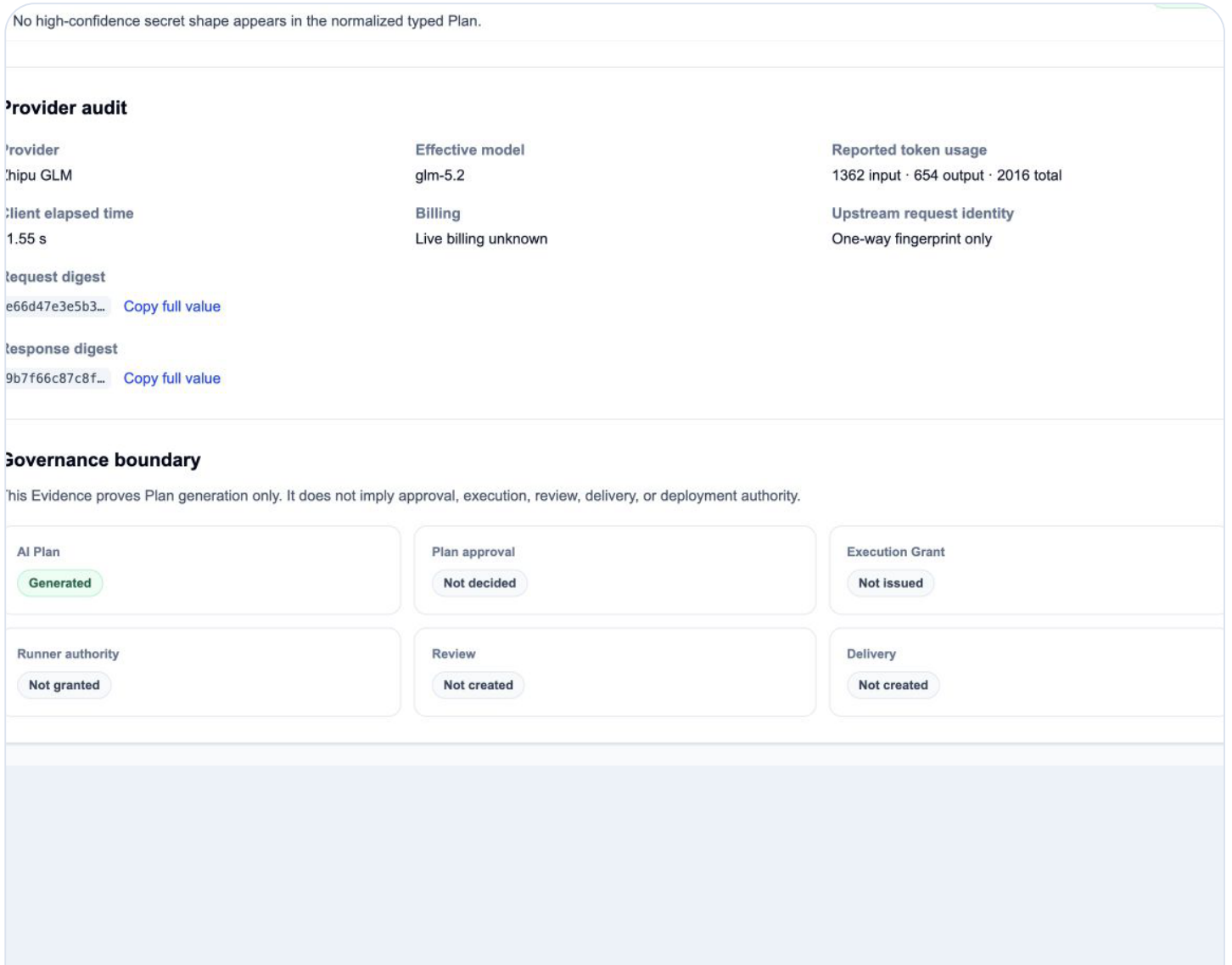
Provider 11,543 ms • client total 11,553 ms

BILLING

live_billing_unknown

不主张金额或账单确定性

lifecycle • succeeded



FAILURE ENGINEERING • 失败工程

派发后的未知状态是事实，不应该被粉饰成简单“失败”。

为什么不自动 retry？因为请求可能已经到达可计费或有副作用的外部系统。后续尝试必须是新的、受控事件。

ATTEMPT 01

Transport ambiguity

请求越过 dispatch 后，本地无法确定 Provider 最终结果。

`outcome_unknown`

ATTEMPT 02

Ambiguity again

保持不可变历史，不把月度用量汇总强行归因到单个 Invocation。

`no auto retry`

ATTEMPT 03

Response-path diagnosis

新增 observability 后，故障被缩小到响应解码路径。

`provider_response_decoding_failed`

ATTEMPT 04

Independent success

做最小 Zhipu compatibility remediation 后，第四次受治理调用成功。

`succeeded`

Safe transport observability → **Response classification** → **Zhipu-specific narrow fix**

RUNNER + WORKSPACE ISOLATION

模型绝不获得 unrestricted shell 权限。

CONTROL PLANE

封闭任务信封

持久化 lifecycle 与 scope。

RUNNER

Outbound consumer

验证 typed envelope，不接受模型任意命令。

WORKSPACE

Exact revision

Runner 持有、隔离、可审计。

GRANT

独立范围授权

进入 mutation path 前的必要条件。

CONCURRENCY

Lease + fencing

避免竞争性 claim / settle。

CLOSURE

Validation + Review

固定 profile 与证据化结果。

源仓库、remote Git、merge、release、deploy 权限均不在默认演示执行路径内。

边界上保留的控制

- ✓ 封闭 typed operations，而非任意命令
- ✓ exact-revision isolated workspace
- ✓ 独立 Runner identity + outbound protocol
- ✓ lease / fencing 保护任务结算
- ✓ fixed validation profile + bounded evidence
- ✓ Provider 无直达 Runner authority 路径

DESIGN PRINCIPLE

执行权由“类型 + 范围 + 身份”共同限制。

POSTGRESQL CONCURRENCY FAILURE

修复原子性边界，而不是用 retry 掩盖 half-write。

RED BASELINE

过早 commit 暴露局部状态

Runner Job 在 child Validation Run 绑定完成前提交，第二个事务可见部分状态，并与子行竞争。

事务 A
部分 child / job 已可见

↔

事务 B
并发进入 child row 竞争

10 / 10 deterministic deadlocks

SQLSTATE 40P01

CONTROLLED REMEDIATION

把相关状态放回一个原子事务

Runner creation 改为非提交创建；Validation Run、steps、permission envelope、Runner Job/event 与最终绑定在一个事务中提交。

Outer transaction
绑定前不暴露半完成状态

→

Concurrent caller
在原子边界等待

10 / 10 controlled passes

0 unbound child records

失败先 rollback，再进入受控 recovery；不用 blind retry 掩盖 exactly-once / transactional visibility 问题。

EVIDENCE-FIRST ENGINEERING

完成声明由 Evidence ledger 支撑，而不是一句“绿色”。

CLAIM → EVIDENCE

版本身份
Git SHA / tree / parent

行为验证
CI / tests / browser Evidence

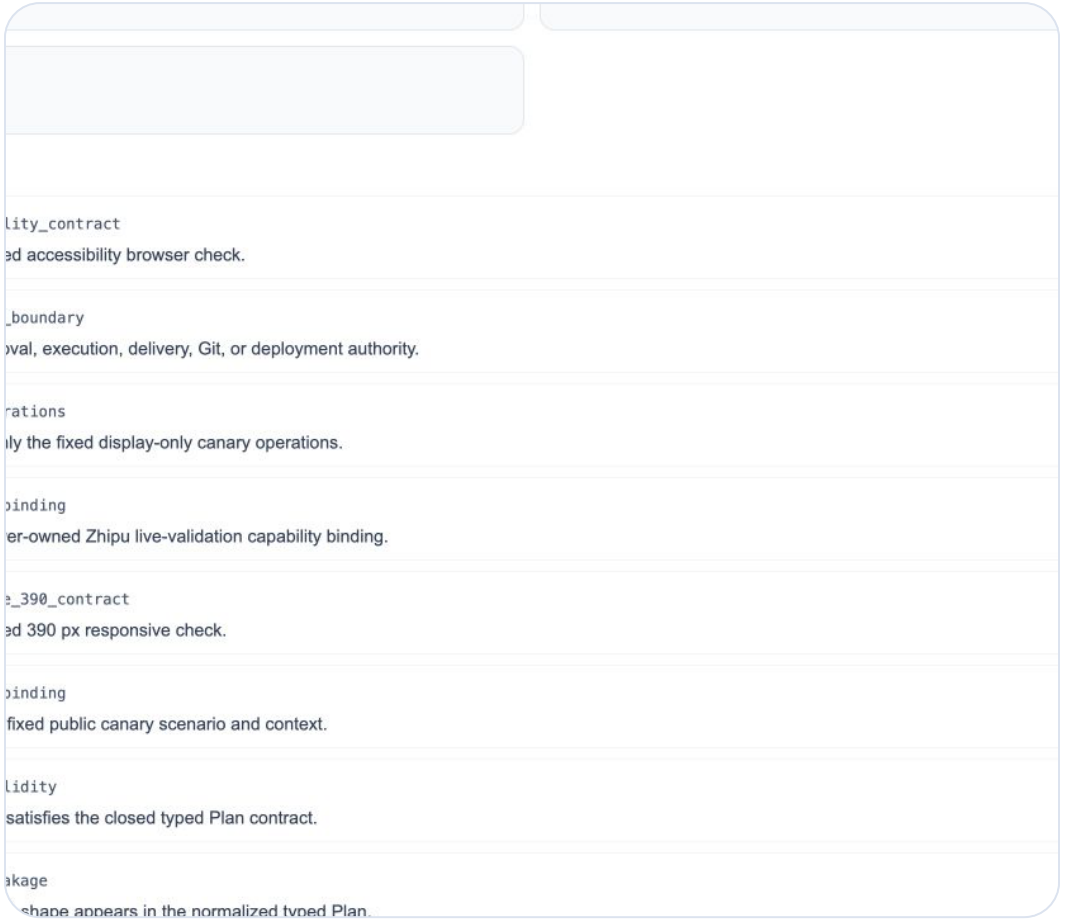
持久状态
PostgreSQL revision / invariants

外部调用
Provider lifecycle / usage / latency

安全边界
Secret-negative / public-safe projection

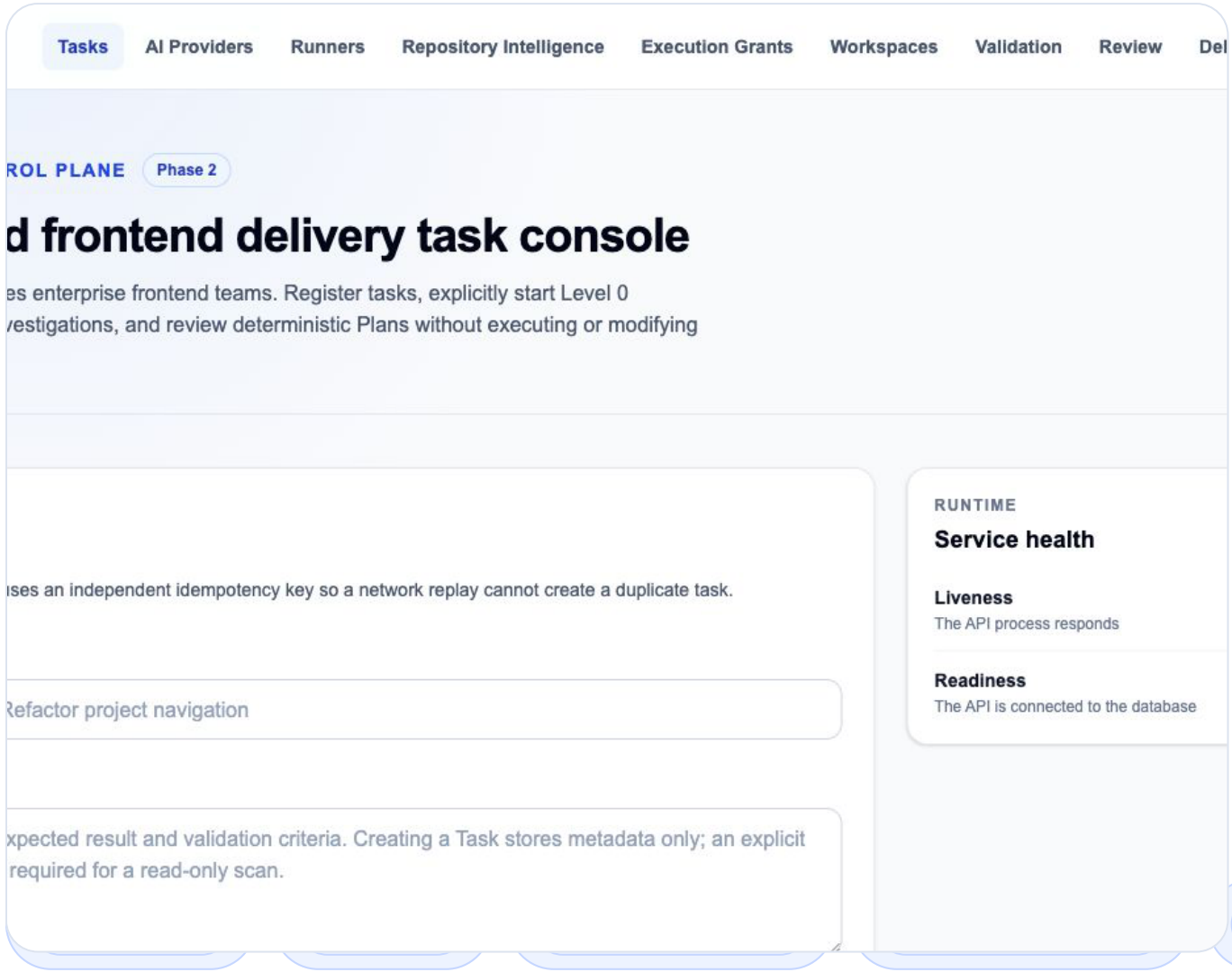
历史事实
failure → root cause → remediation

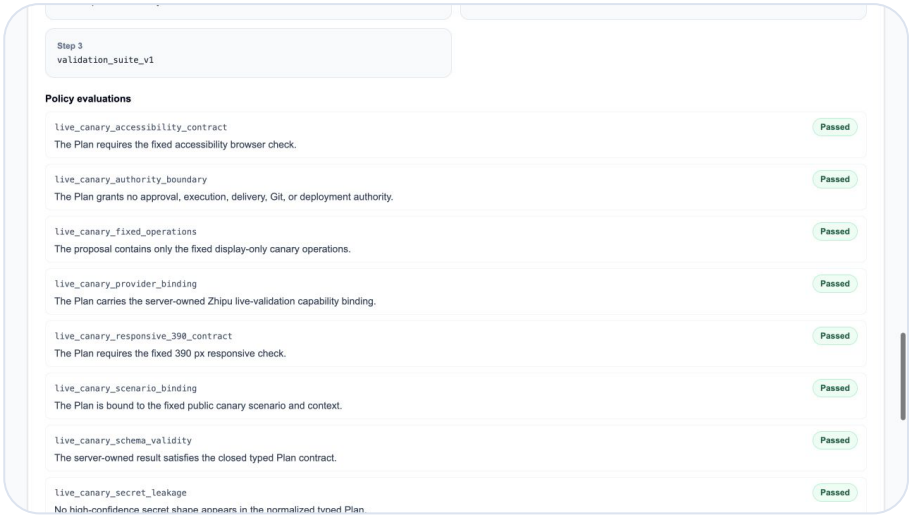
FACT / INFERENCE / NOT VERIFIED 严格区分；未知就保持未知。

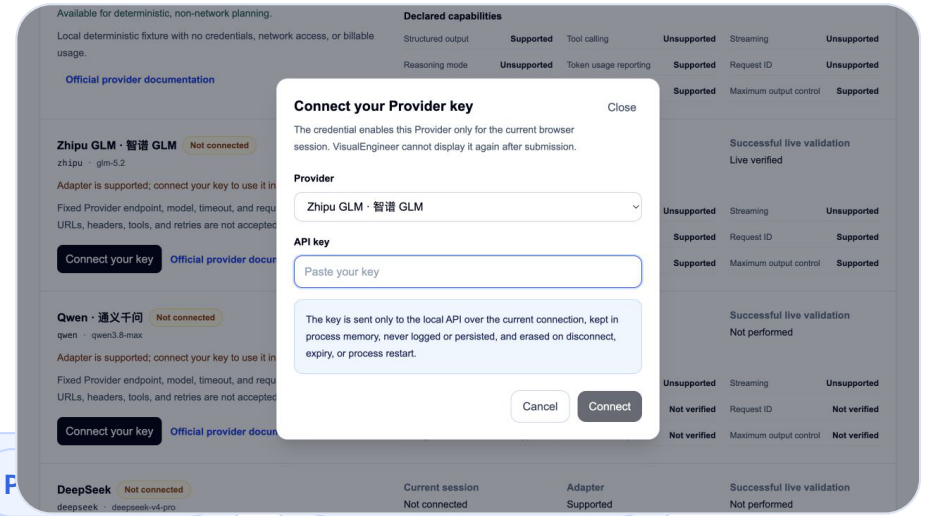


PRODUCT EXPERIENCE · 产品体验

让治理边界在浏览器里可见，而不是藏在后端文档里。







ENGINEERING STACK · 技术栈

从 Browser UI 到持久化状态与 Evidence 的类型化边界。

WEB CONSOLE

Frontend

Next.js App Router
React
TypeScript strict
Tailwind CSS
next-intl

CONTROL PLANE

Backend

Python
FastAPI
Pydantic
SQLAlchemy
Alembic

STATE + EXECUTION

Runtime

PostgreSQL
Independent Runner
typed task protocol
isolated fixture workspace

QUALITY + EVIDENCE

Validation

Vitest
Playwright
axe
Ruff / mypy
Git + CI Evidence

技术栈不是“关键词堆叠”：前端 TypeScript、API 边缘契约、typed Python 服务与 PostgreSQL 约束共同把每条安全边界具体化。

MY ROLE + AI-ASSISTED ENGINEERING

AI 加速交付；产品、架构、风险与验收责任仍由人承担。

I OWNED

产品定义

问题、工作流、界面优先级、范围与验收标准。

架构与边界

权限模型、生命周期规则、typed contracts、风险决策。

验证与证据

Claims-to-Evidence、测试预期、失败解释、最终验收。

面试可解释性

能够讨论架构、状态机、真实故障、取舍与限制。

CHATGPT + CODEX ACCELERATED

在明确约束与审查下加速工程执行

AI 主要用于仓库调研、实现、测试、failure diagnosis、文档、Git / CI 与 Evidence 整理。

WHAT I DO NOT CLAIM

不声称每一行代码都由人工手写，也不把安全边界、风险决策、验证与最终验收的责任转移给模型。

> 项目价值来自：我能够解释“为什么这样设计、哪里失败过、怎么验证、为什么不走更危险的捷径”。

SCOPE + NEXT CONVERSATION

清晰的边界，比夸大的声明更适合作为面试作品。

VisualEngineer 的重点不是“让 AI 自动写更多代码”，而是把 Planning、Authority、Execution 和 Evidence 做成可以解释、可以验证的系统边界。

公开 GitHub

github.com/dankk0220abc-prog/visual-engineer

扫码查看 README、代码、CI 与 Portfolio PDF

CURRENT SCOPE

- 主开发已完成；封存证据包含一次成功 Zhipu / GLM-5.2 Live Validation。
- 其他 Provider adapter 不被表述为已完成 Live Validation。
- Fixture 是最安全、可复现、无真实计费的 Demo 模式。
- 不主张生产部署、客户、收入、用户规模或未验证性能收益。
- BYOK 为会话范围易失性处理，不夸大为商业级多租户 Secret Vault。

面试可继续深入：Provider lifecycle · PostgreSQL invariants · Runner boundary · BYOK · Evidence strategy